TEAM PROJECT REPORT

Assignment 6

jQuery in BookingApp

A continuation of the existing hotel-booking frontend, adding live search, interaction feedback and shared utilities.

Team: BookingApp

Group: SE-2513

Members: Allazhar Aldongarov, Dias Serikov, Yerkenov Abylay

OBJECTIVE

Use jQuery to simplify DOM updates, event handling and animations while retaining the existing responsive pages and booking flow.

SCOPE AND EVIDENCE

Tasks 0-9 are documented on the next ten pages. Every task includes implementation steps, an image of an excerpt read directly from the final source files, and an actual browser screenshot. The last page records validation and implementation lessons.

PROJECT LINKS

Shared repository:
https://github.com/jjestar/web

Deployed website:
https://jjestar.github.io/web/

The forms simulate frontend actions. They do not send email, take payment or create a real reservation. Use invented guest details for demonstrations.

TASK 0

jQuery setup

Shared setup across all six HTML pages

Load jQuery before dependent scripts and run page initialization after the DOM is ready.

1. Place the jQuery 4.0.0 CDN script in the head. Use defer to keep the declared script order.

2. Load common.js and the relevant page script after jQuery; keep JavaScript in external files.

3. Use $(document).ready() and print the required message. Check the actual browser console.

SOURCE CODE EXCERPT

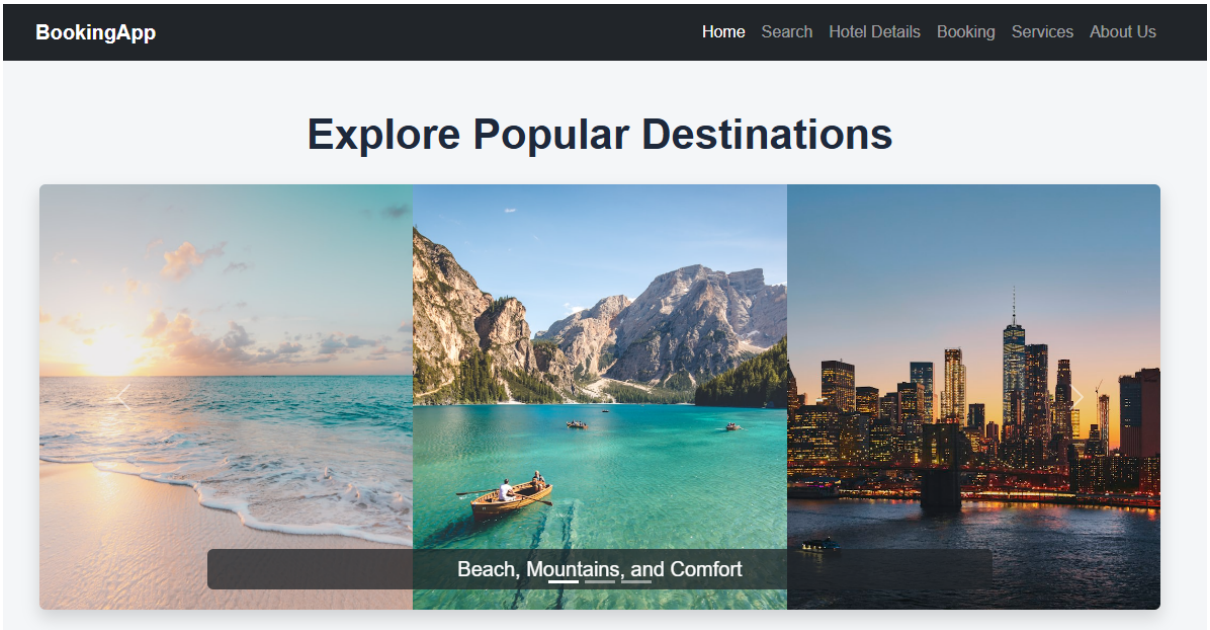
index.html | lines 7-13

```
  7  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
     rel="stylesheet">
  8  <link rel="stylesheet" href="css/allstyle.css">
  9  <link rel="stylesheet" href="css/common.css">
 10  <script src="https://code.jquery.com/jquery-4.0.0.min.js" defer></script>
 11  <script src="js/common.js" defer></script>
 12  <script src="js/script.js" defer></script>
 13  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"
     defer></script>
```

js/script.js | lines 1-2

```
  1  $(document).ready(function () {
  2      console.log('jQuery is ready!');
```

BROWSER RESULT



Home page loaded with the final shared script setup. Console evidence is recorded below.

Captured console output: jQuery is ready!
Runtime library: jQuery 4.0.0 (verified in the browser)

Demonstration check. Browser evidence: jQuery 4.0.0 and the message "jQuery is ready!". The submitted website remains a frontend demonstration.

TASK 1

Real-time search and live filter

Search results | search-results.html

Update the hotel list immediately while the visitor types, without reloading the page.

1. Read the search input and normalize the query with trim() and toLowerCase().

2. Filter hotel name, location and description together with the existing rating and amenity controls.

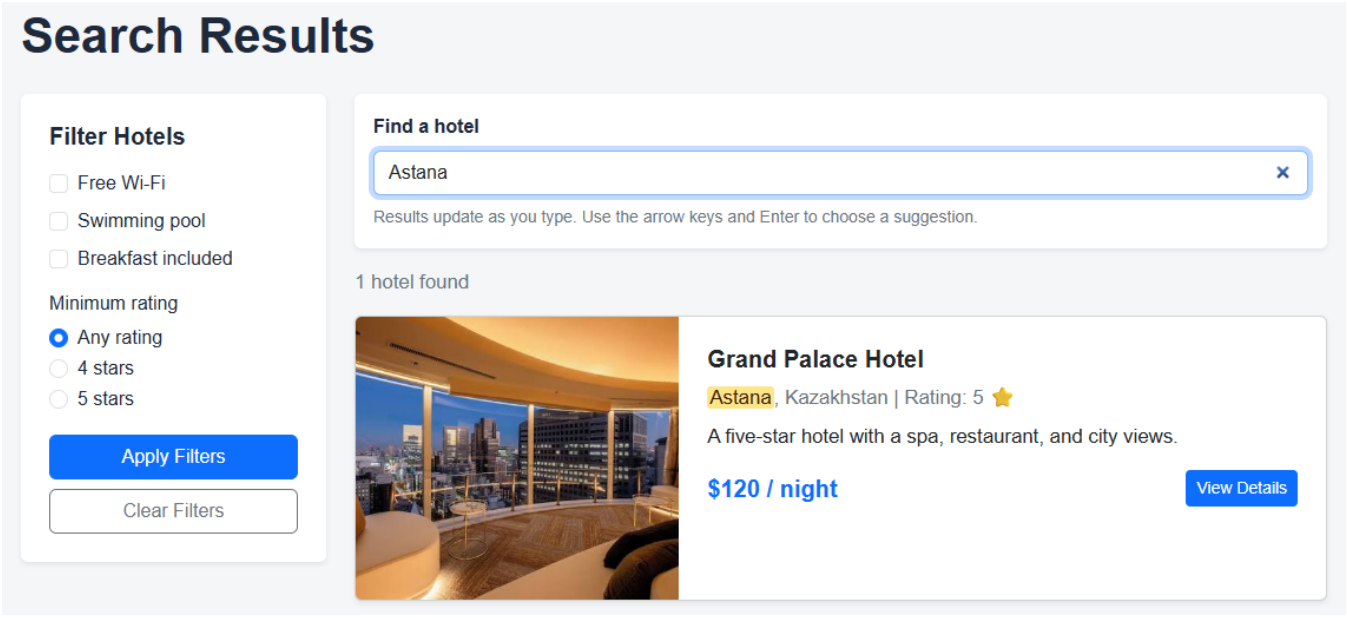
3. Render the matching cards and result count on the input event. Show a clear empty-results message.

SOURCE CODE EXCERPT

```
js/script.js  |  lines 195-204
195  const filteredHotels = hotelsDatabase.filter(function (hotel) {
196      const searchableText = [hotel.name, hotel.location, hotel.description].join('
     ').toLowerCase();
197      return searchableText.includes(lowerQuery) &&
198          (rating === 'all' || hotel.rating >= Number(rating)) &&
199          (!needsWifi || hotel.wifi) &&
200          (!needsPool || hotel.pool) &&
201          (!needsBreakfast || hotel.breakfast);
202  });
203
204  renderHotels(filteredHotels, query);

js/script.js  |  lines 259-262
259  $searchInput.on('input', function () {
260      applyFilters();
261      updateSuggestions();
262  });
```

BROWSER RESULT



A typed query narrows the hotel list. Existing filters remain available beside the results.

Demonstration check. Demonstrate a matching query, mixed-case input, no matches, and Clear. Search and checkbox/rating filters combine.

TASK 2

Autocomplete suggestions

Search results | search-results.html

Offer useful hotel-name and destination suggestions while the visitor enters a query.

1. Collect hotel names and locations from hotelsDatabase, then remove duplicates with Set.

2. Keep matching choices and show at most six options in the dropdown.

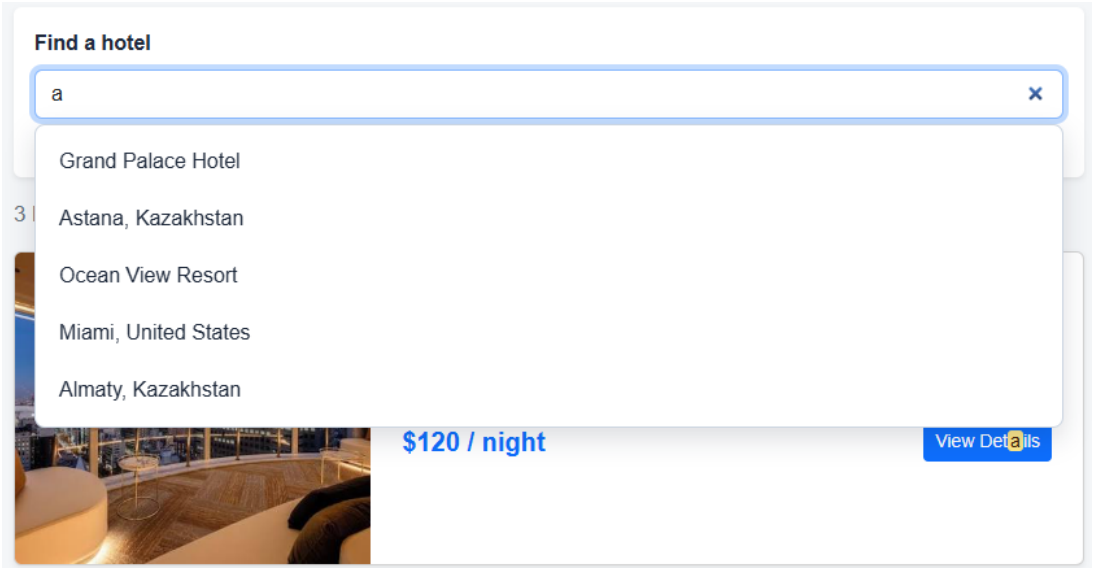
3. A click or Enter inserts the chosen text and refreshes results. Arrow keys select; Escape closes.

SOURCE CODE EXCERPT

```
js/script.js  |  lines 227-238
227  const matches = [...new Set(choices)].filter(function (choice) {
228      return choice.toLowerCase().includes(query);
229  }).slice(0, 6);
230
231  matches.forEach(function (choice, index) {
232      $('<li>')
233          .attr('id', 'search-option-' + index)
234          .attr('role', 'option')
235          .attr('aria-selected', 'false')
236          .text(choice)
237          .appendTo($suggestions);
238  });

js/script.js  |  lines 246-250
246  function chooseSuggestion(option) {
247      $searchInput.val($(option).text());
248      closeSuggestions();
249      applyFilters();
250      $searchInput.trigger('focus');
```

BROWSER RESULT



The suggestion list is built from the actual hotel data and appears below the search field.

Demonstration check. Try a partial hotel name, select by mouse, then repeat with Arrow Down and Enter. A blank query hides the list.

TASK 3

Search highlighting

Hotel result cards | js/script.js

Make every matching occurrence in the result-card text easy to see.

1. Keep the original text and find matches in a lowercase copy, while preserving the displayed spelling.

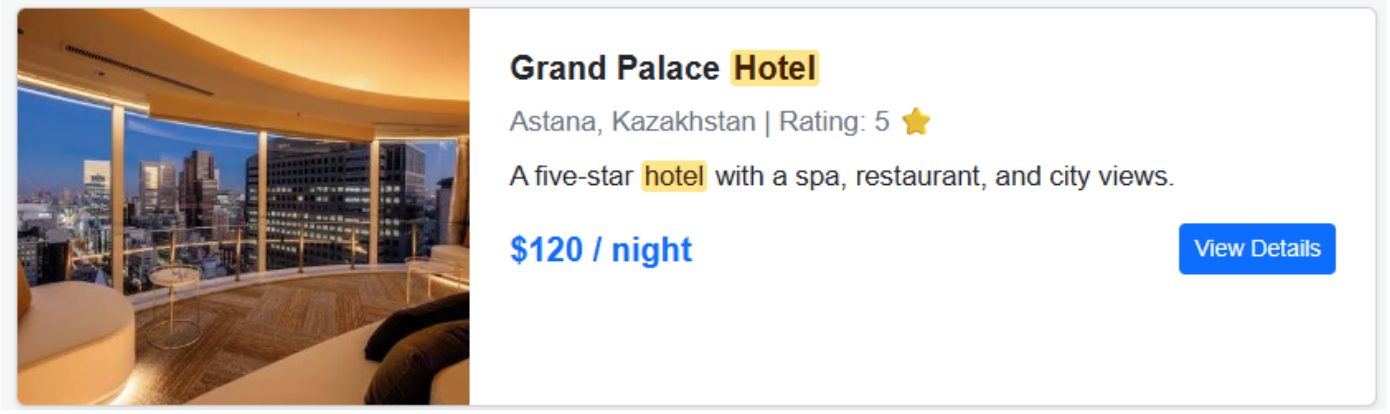
2. Append ordinary text as text nodes and matching segments as mark elements.

3. Apply highlighting after each render. Clearing the query recreates clean, unhighlighted cards.

SOURCE CODE EXCERPT

```
js/script.js  |  lines 128-142
128  let match = lowerText.indexOf(lowerQuery);
129
130  if (!query || match === -1) {
131      return;
132  }
133
134  $(element).empty();
135
136  while (match !== -1) {
137      element.appendChild(document.createTextNode(originalText.slice(start, match)));
138      $('<mark>').text(originalText.slice(match, match + query.length)).appendTo(element);
139      start = match + query.length;
140      match = lowerText.indexOf(lowerQuery, start);
141  }
142
```

BROWSER RESULT



Matching text is highlighted in the displayed results; the original content remains readable.

Demonstration check. Repeated matches are marked. Literal characters such as brackets are treated as text, not as a regular expression or HTML.

TASK 4

Stylized scroll progress bar

About and Services | js/about-services.js

Show how far the visitor has scrolled through the current page.

1. Create a fixed progress track and colored fill using jQuery; style them in ablstyle.css.

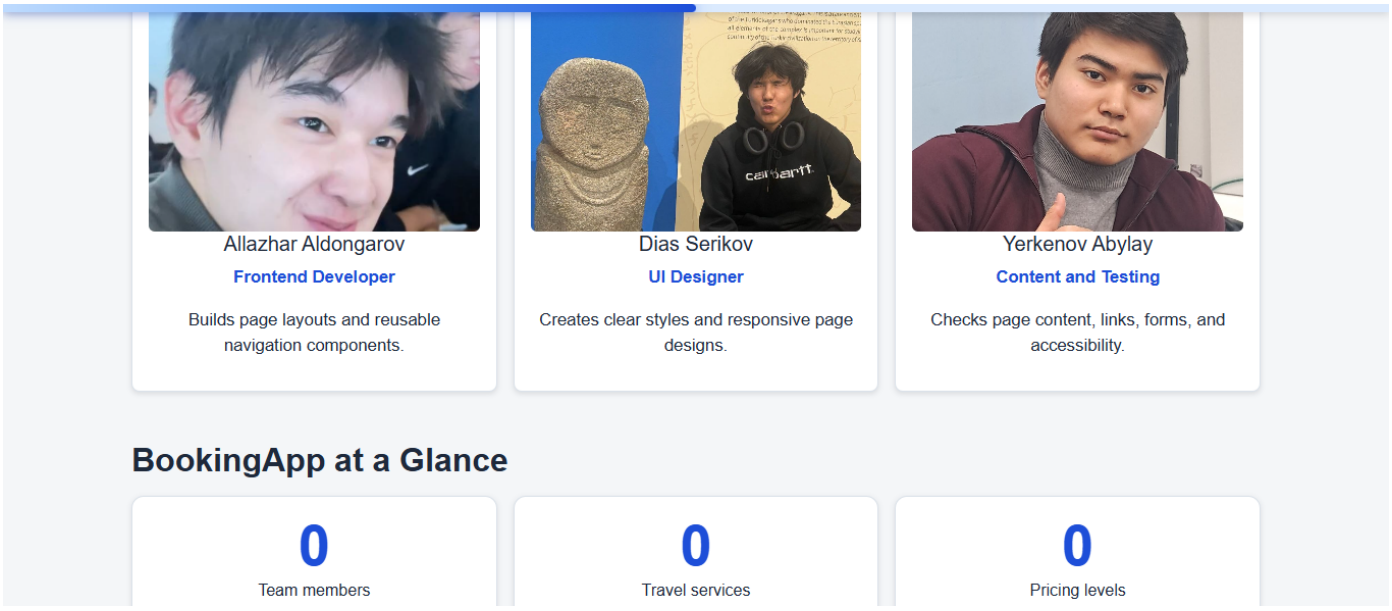
2. Calculate scroll position divided by the scrollable document height, then convert it to a percentage.

3. Update fill width and aria-valuenow on scroll and resize. Clamp the result between 0 and 100.

SOURCE CODE EXCERPT

```
js/about-services.js  |  lines 226-234
226  function initializeScrollProgress() {
227      const $bar = $('<div class="scroll-progress-track" role="progressbar" aria-label="Page
     scroll progress" aria-valuemin="0" aria-valuemax="100" aria-valuenow="0"><div class="scroll-
     progress-fill"></div></div>').prependTo('body');
228      $(window).on('scroll resize', function () {
229          const max = $(document).height() - $(window).height();
230          const percent = max > 0 ? Math.min(100, Math.max(0, window.scrollY / max * 100)) :
     0;
231          $bar.children().css('width', percent + '%');
232          $bar.attr('aria-valuenow', Math.round(percent));
233      }).trigger('scroll');
234  }
```

BROWSER RESULT



The colored bar stays at the top of the viewport while the content scrolls.

Demonstration check. The bar reaches 100% at the bottom. A page without scrollable height avoids division by zero.

TASK 5

Animated number counter

About | BookingApp at a Glance

Animate the statistics once when the visitor scrolls to their section.

1. Store each final value in data-count and begin with the visible value 0.

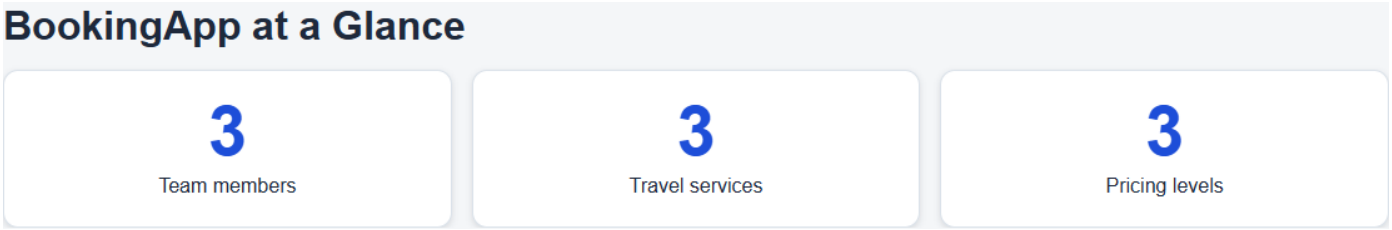
2. Use IntersectionObserver to start jQuery animate() when the statistics become visible.

3. Update the displayed integer during the animation and set the exact target at completion. Disconnect the observer.

SOURCE CODE EXCERPT

```
js/about-services.js  |  lines 239-251
239  const observer = new IntersectionObserver(function (entries) {
240      if (!entries[0].isIntersecting) return;
241      const duration = window.matchMedia('(prefers-reduced-motion: reduce)').matches ? 0 :
     1400;
242      $('.stat-number', section).each(function () {
243          const $number = $(this);
244          const target = Number(this.dataset.count);
245          $({ value: 0 }).animate({ value: target }, {
246              duration: duration,
247              step: value => $number.text(Math.floor(value)),
248              complete: () => $number.text(target)
249          });
250      });
251      observer.disconnect();
```

BROWSER RESULT



The three counters describe the actual project: team members, travel services and pricing levels.

Demonstration check. Counters finish at 3, 3 and 3 and do not restart on every scroll. Reduced-motion preferences skip the animation.

TASK 6

Loading spinner on submit

Contact form and service-cost estimator

Provide visible waiting feedback and prevent duplicate submission during a simulated request.

1. Validate the form and capture its submitted values before starting the waiting state.

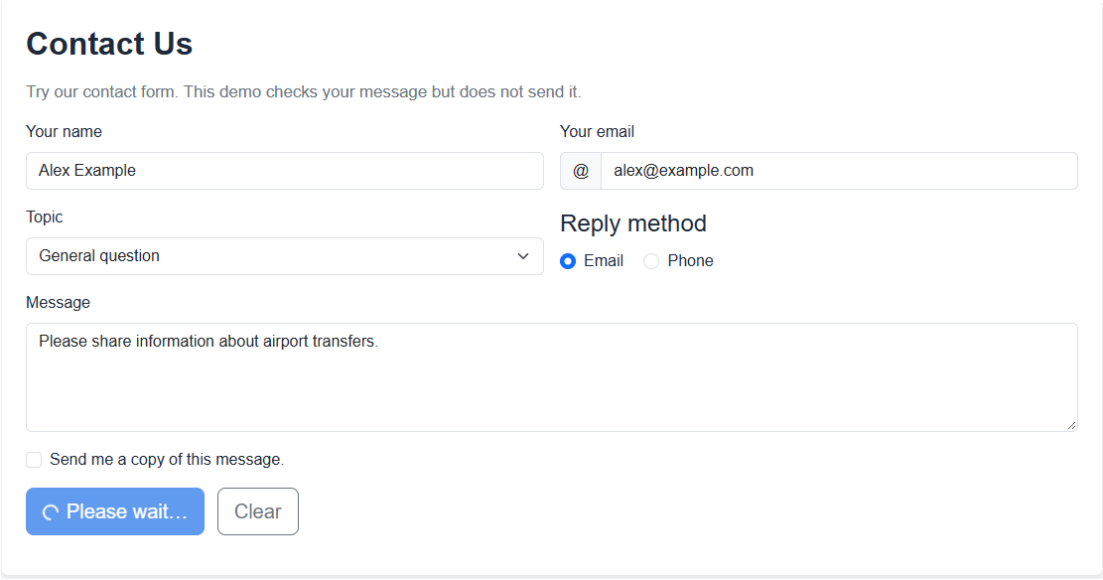
2. Show a spinner and "Please wait...", disable Submit and set aria-busy for 1,200 milliseconds.

3. Restore the button and display the result from captured values after the delay. Reset cancels the pending timeout and restores the form state.

SOURCE CODE EXCERPT

```
js/about-services.js  |  lines 274-289
274      button.innerHTML = '<span class="submit-spinner" aria-hidden="true"></span>Please
     wait…';
275      button.disabled = true;
276      form.setAttribute('aria-busy', 'true');
277
278      pendingTimeout = setTimeout(function () {
279          pendingTimeout = null;
280          restoreButton();
281          complete(values);
282      }, 1200);
283  });
284
285  $(form).on('reset', function () {
286      clearTimeout(pendingTimeout);
287      pendingTimeout = null;
288      restoreButton();
289  });
```

BROWSER RESULT



A valid contact submission shows the disabled waiting button before the demo result appears.

Demonstration check. Changing fields during the wait does not change the submitted values. Clear cancels the result; invalid fields do not start the spinner.

TASK 7

Temporary notification system

Shared helper | js/common.js

Confirm user actions with a short notification that disappears automatically.

1. Create one accessible notification container and reuse it for subsequent actions.

2. Set the message with text(), then use fadeIn(), delay() and fadeOut() for its lifetime.

3. Call the helper after demo subscription, contact, service calculation and booking actions. Stop previous animation queues first.

SOURCE CODE EXCERPT

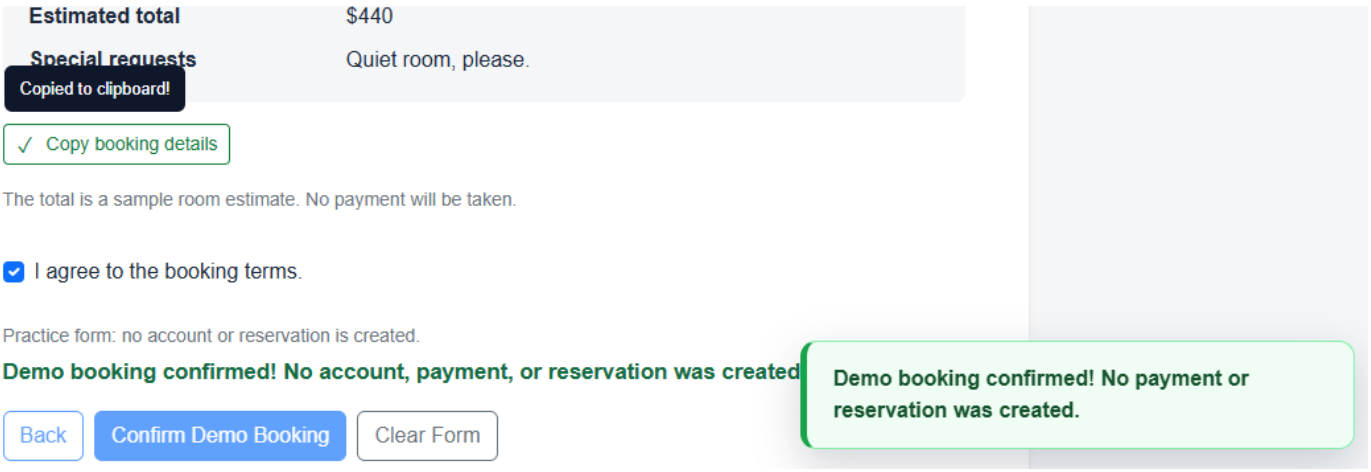
js/common.js | lines 1-18

```
function showNotification(message, type = 'success') {
    let notification = $('#notification');

    if (notification.length === 0) {
        notification = $('<div>', {
            id: 'notification',
            class: 'app-notification',
            role: 'status',
            'aria-live': 'polite',
            'aria-atomic': 'true'
        }).appendTo('body');
    }

    notification.stop(true, true);
    notification.toggleClass('notification-error', type === 'error');
    notification.text(message);
    notification.fadeIn(180).delay(2600).fadeOut(350);
}
```

BROWSER RESULT



A successful demo action produces a temporary status message with relevant wording.

Demonstration check. The notification fades out after its delay. Repeated actions replace its text instead of creating a stack of old messages.

TASK 8

Copy to clipboard button

Hotel location and booking review

Copy useful text and clearly confirm whether the operation succeeded.

1. Use a data-copy-target selector to find the text belonging to the clicked Copy button.

2. Await navigator.clipboard.writeText() in a secure context; fall back to selected text when needed.

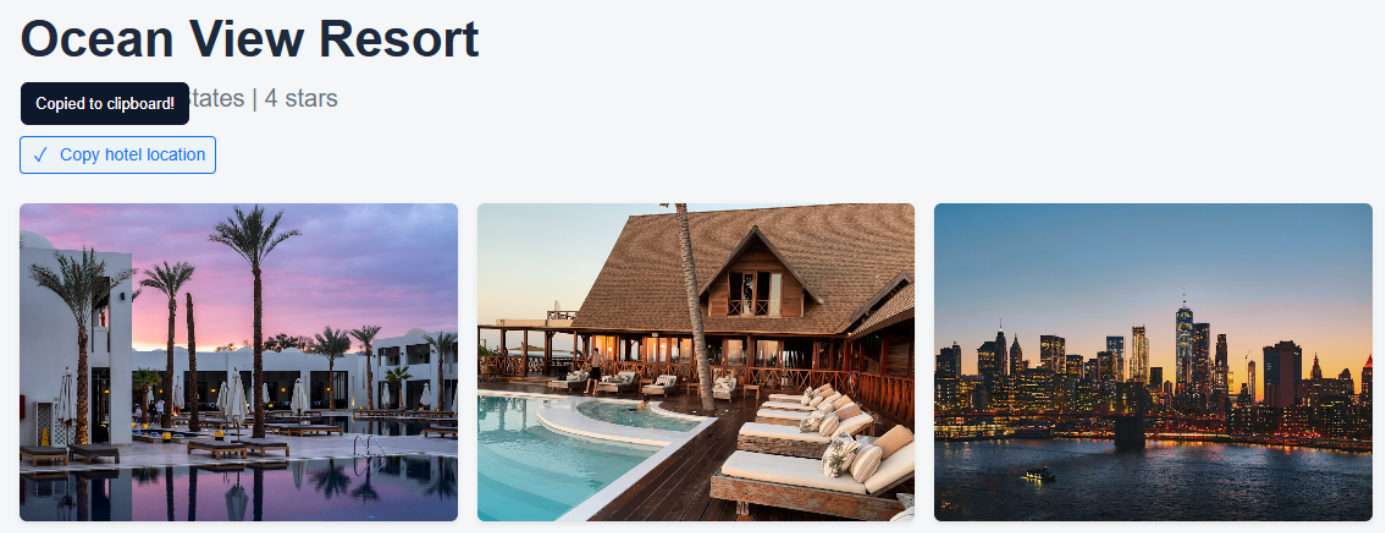
3. Only after success, show a checkmark and "Copied to clipboard!" tooltip. Restore the original icon after 2.2 seconds.

SOURCE CODE EXCERPT

```
js/common.js  |  lines 52-61
 52  async function copyTextToClipboard(text) {
 53      if (navigator.clipboard && window.isSecureContext) {
 54          try {
 55              await navigator.clipboard.writeText(text);
 56              return;
 57          } catch (error) {
 58              return copyTextWithSelection(text);
 59          }
 60      }
 61

js/common.js  |  lines 118-122
118  try {
119      await copyTextToClipboard(text);
120      icon.text('✓');
121      tooltip.text('Copied to clipboard!').prop('hidden', false);
122      button.addClass('copy-success');
```

BROWSER RESULT



The location Copy button displays its success icon and tooltip after the clipboard operation.

Demonstration check. Paste into a text field to verify the copied value. Failure shows an error rather than an incorrect success message.

TASK 9

Image lazy loading

Home destinations and dynamically rendered hotel results

Request deferred images only when their elements enter the visible part of the page.

1. Start each deferred image with a local placeholder and keep the real file path in data-src.

2. Use jQuery scroll position, height and offset values to test intersection with the viewport.

3. Move data-src to src for visible images, then remove data-src. Run on ready, scroll, resize and after rendering new hotel cards.

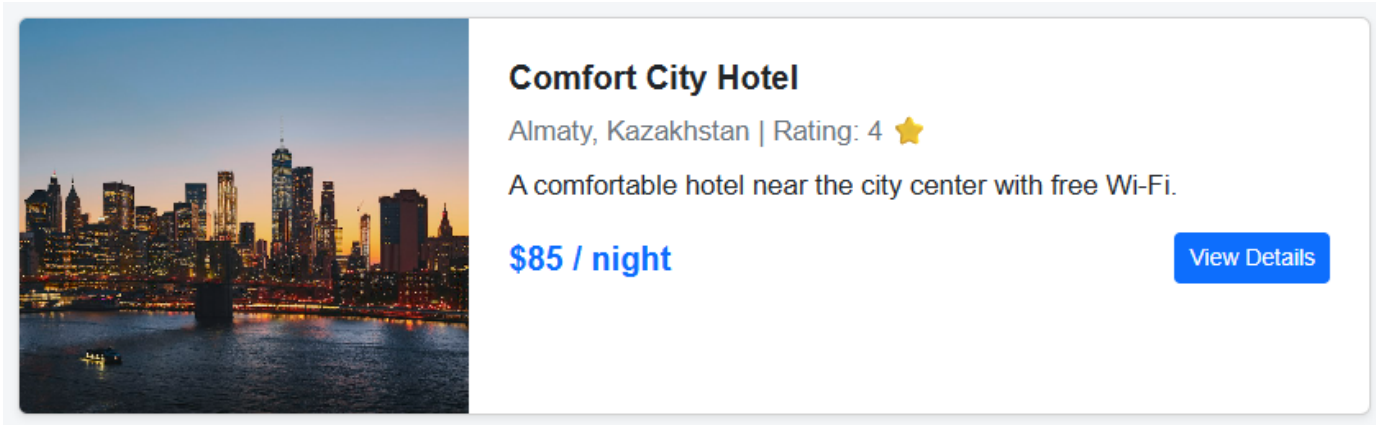
SOURCE CODE EXCERPT

```
js/common.js  |  lines 20-22
 20  function loadVisibleImages() {
 21      const windowTop = $(window).scrollTop();
 22      const windowBottom = windowTop + $(window).height();

js/common.js  |  lines 30-35
 30
 31  const imageTop = image.offset().top;
 32  const imageBottom = imageTop + image.outerHeight();
 33
 34  if (imageTop < windowBottom && imageBottom > windowTop) {
 35      const imagePath = image.attr('data-src');

js/common.js  |  lines 46-50
 46              image.removeAttr('data-src');
 47              image.attr('src', imagePath);
 48          }
 49      });
 50  }
```

BROWSER RESULT



The lower hotel-result image has loaded after scrolling. Network evidence confirms that its file request was deferred.

Demonstration check. Observed on desktop: destination-3.jpg was absent from initial requests and appeared after scrolling. The same behavior was checked at mobile width.

Validation and reflection

VALIDATION RESULTS

53/53 checks passed. Headless Chrome covered all six pages at desktop width (1360 px) and mobile width (390 px). No JavaScript runtime errors, broken visible images or horizontal overflow were recorded.

Setup: External jQuery 4.0.0 loads before dependent scripts; ready messages are visible in the browser console.

Search: Live filtering, combined controls, no-results state, suggestions, keyboard selection and literal-text highlighting.

Feedback: Progress reaches the bottom; counters finish correctly; spinner captures submitted values and reset cancels the pending result.

Shared utilities: Temporary notifications, clipboard success and reset, and deferred image requests on scroll.

Regression: Existing page navigation, hotel detail selection and the multistep demo booking remain available.

CHALLENGES AND SOLUTIONS

Combining old and new search behavior. Use one applyFilters() function so the query, star rating and amenities stay consistent.

Safe highlighting. Use text nodes and mark elements instead of inserting the search query as HTML or a regular expression.

Changing forms during a simulated request. Capture validated values at submit time. Clear the timeout when the form is reset.

Dynamic image cards. Run the shared visibility check again after renderHotels() adds new image elements.

Clipboard permissions. Await the copy operation, provide a fallback and show success only after a confirmed copy.

REFLECTION

The main improvement is consistent feedback: visitors can see search changes immediately, understand waiting states and confirm actions. Shared helpers reduce duplicated behavior. The project remains a static frontend, so the simulated actions are explicitly labelled as demonstrations.

DEPLOYMENT

The updated source was committed to the shared repository and successfully deployed with GitHub Pages on 2 November 2026 (source commit 720e143). Live-site checks passed 8/8: all six pages returned HTTP 200, jQuery 4.0.0 loaded, search and clipboard worked, deferred image state and a notification were observed, and no JavaScript errors were recorded.

Repository: github.com/jjestar/web
Website: jjestar.github.io/web/